

stacked

Stacked cubes and dices

v0.1.0

Akilon <<https://github.com/Akilon27>>

MIT

About

Cube stacks were introduced into the French middle-school curriculum in 2025, which is what motivated the creation of this package.

I was inspired by `tikz3d-fr` (author: Cédric Pierquet) and by part of `ProfCollege` for LaTeX (author: Christophe Poulain), but this implementation is not a copy of either. I used AI (Gemini and Claude) for the dice and the random part of the `pile` function; the rest is my own work, except for the `dice-face` function, which was created by `eric1102` (on Discord) and is included here with his permission.

Functions: cube stacks and dice

- `cube()`
- `pile()`
- `building()`
- `above-view()`
- `right-view()`
- `front-view()`
- `below-view()`
- `left-view()`
- `back-view()`
- `thetaphi()`
- `dice()`
- `throws()`
- `dice-face()`

cube

CeTZ command for a cube.

Parameters

```
cube(  
  a: array ,  
  c: auto color array ,  
  border-stroke: boolean color ,  
  thickness: int float ratio ,  
  light: float ratio array ,  
  side: boolean  
) -> function
```

a array

Position

c `auto` or `color` or `array`

auto → (red on the right, green on top, yellow on the left), otherwise one or 3 colors

Default: `auto`

border-stroke `boolean` or `color`

Lines in black or in the cube color, default false

Default: `false`

thickness `int` or `float` or `ratio`

Stroke thickness (from 1pt), default 100%

Default: `100%`

light `float` or `ratio` or `array`

How much to lighten when using a single color, 20% by default

Default: `20%`

side `boolean`

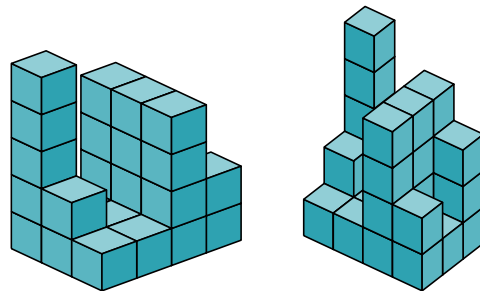
Internal parameter for the side view with pile

Default: `false`

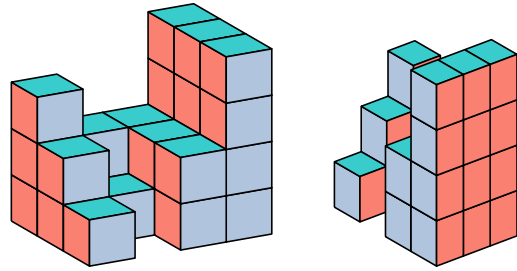
pile

Stack of small cubes on a grid, given as a list from back to front 3251, then from left to right (separated by commas). A color can be specified at the beginning or the end. With the default colors using none: front → yellow, top → green, right → red. Alternatively, a list of 3 colors can be provided with colors.

```
#pile(3415,1412,2411,eastern,side:true,  
iso:1,draw-scale:.6,border-stroke:true)
```

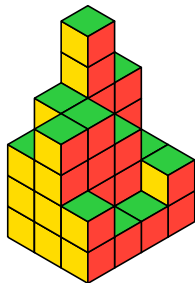


```
#pile(4223,4212,4201,colors:
(rgb("#fa8072"),rgb("#b0c4de"),teal),
side: true,iso: 2,draw-scale: .75)
```

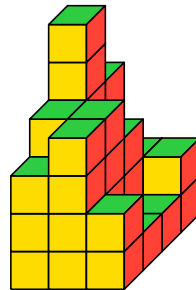


```
#grid(columns: 5*(1fr,), column-gutter: 1.5em,
..for i in (true,false,1,2,3) {[== iso: #i
#pile("4643",2344,2112, iso: i)
],,)}
)
```

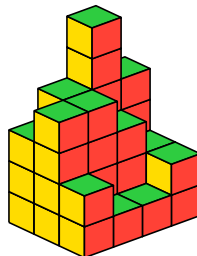
iso: true



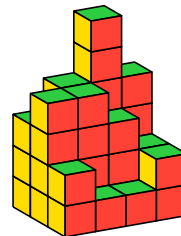
iso: false



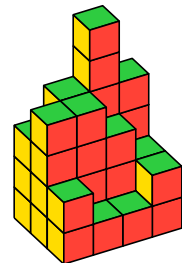
iso: 1



iso: 2

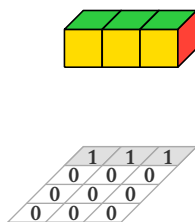


iso: 3

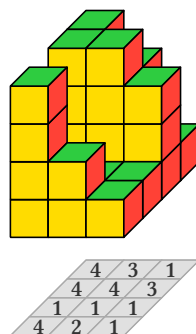


```
#grid(columns: 5*(3.2cm,), column-gutter: 2em,
..for i in (1,2,3,4) {[== random-mode: #i
#pile(random:true, random-mode: i,shadow:true,seed:2*i,plan:true)
],,)}
)
```

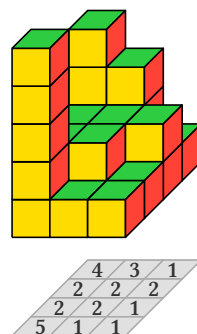
random-mode: 1



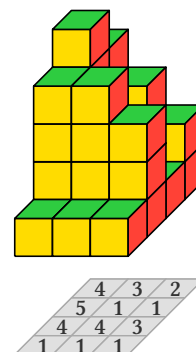
random-mode: 2



random-mode: 3



random-mode: 4



z:0 for front view, y:0, z:(0,-1) for top view, x:0, z:(-1,0) for right view

Parameters

```
pile(  
  ..a: int str array arguments ,  
  random: boolean ,  
  width: int ,  
  depth: int ,  
  height: int ,  
  seed: int ,  
  plan: boolean ,  
  labels: boolean ,  
  x: array ,  
  y: array ,  
  z: array ,  
  thickness: int float ratio ,  
  border-stroke: boolean color ,  
  side: boolean ,  
  light: float ratio array ,  
  draw-scale: float ratio ,  
  x-spread: float int ,  
  y-spread: float int ,  
  z-spread: float int ,  
  colors: array ,  
  grids: boolean ,  
  solution: boolean ,  
  below-separation: auto int float ,  
  left-separation: auto int float ,  
  back-separation: auto int float ,  
  iso: boolean int ,  
  space: length relative length fraction ,  
  holes: boolean ,  
  random-mode: auto int ,  
  views: array ,  
  views-names: array ,  
  axes: boolean ,  
  opacity: ratio ,  
  shadow: boolean ,  
  disposition: str ,  
  box-args: dictionary  
) -> content
```

..a int or str or array or arguments

List of numbers corresponding to the heights back-to-front, left-to-right

random boolean

Enables a random mode: automatically generates the stack (instead of positional arguments), with equal or decreasing heights from row to row

Default: **false**

width `int`

Width (number of slices along X) in random mode

Default: `3`

depth `int`

Depth (number of digits per slice along Z) in random mode

Default: `4`

height `int`

Maximum height (along Y) in random mode

Default: `5`

seed `int`

Seed of the pseudo-random generator for reproducible results: same seed → same stack; change it to obtain another one

Default: `1`

plan `boolean`

Display the level plan (numbered top view)

Default: `false`

labels `boolean`

Display the view labels on the grids

Default: `false`

x `array`

Direction of the first coordinate, default (1,0)

Default: `(1,0)`

y `array`

Direction of the second coordinate, default (0,1)

Default: `(0,1)`

z array

Direction of the third coordinate, default (-0.5,-0.5)

Default: (-.5, -.5)

thickness int or float or ratio

Stroke multiplier (from 1pt), default 60%, i.e. 0.6pt

Default: 60%

border-stroke boolean or color

Lines in black, default false

Default: false

side boolean

Add a second view of the assembly from the “right”

Default: false

light float or ratio or array

How much to lighten when using a single color

Default: 20%

draw-scale float or ratio

Changes the edge size of a base cube

Default: .5

x-spread float or int

Allows the generated stack to be “exploded” to the right to make visualization by “slices” easier

Default: 0

y-spread float or int

Allows the generated stack to be “exploded” upward to make visualization by “layers” easier

Default: 0

z-spread float or int

Allows the generated stack to be “exploded” forward to make visualization by “slices” easier

Default: 0

colors array

Provide a list of 3 colors for the three faces (left, right, top)

Default: ()

grids boolean

Display three grids allowing the student to directly draw the front, top, and left views

Default: false

solution boolean

When set to true, display the associated solid’s front, top, and left views on the grids

Default: false

below-separation auto or int or float

Separation of the bottom grid (auto = automatically calculated)

Default: auto

left-separation auto or int or float

Separation of the left grid (auto = automatically calculated)

Default: auto

back-separation auto or int or float

Separation of the back grid (auto = automatically calculated)

Default: auto

iso `boolean` or `int`

Allows different perspectives:

- `true` = isometric representation,
- `1` = `tikz3d-fr` view,
- `2` = first ProfCollege view,
- `3` = second ProfCollege view,
- otherwise = default CeTZ view

Default: `false`

space `length` or `relative length` or `fraction`

Spacing from the second view

Default: `2em`

holes `boolean`

Allows “holes” (zero height) in the random stack starting from the second stack; the first stack always remains “full” (height ≥ 1 everywhere)

Default: `false`

random-mode `auto` or `int`

Specific random mode:

- `1` (double-min LCG by Gemini),
- `2` and `4` (decreasing LCG inspired by ProfCollege by Claude),
- `3` (suiji package)

Default: `auto`

views `array`

List of views to display among (“top”, “front”, “right”)

Default: (`"dessus"`, `"face"`, `"droite"`)

views-names `array`

Names of the displayed views among (“top”, “front”, “right”)

Default: (`"Vue de dessus"`, `"Vue de face"`, `"Vue de droite"`)

axes `boolean`

Display axis markers (coordinates) on the ground

Default: `false`

opacity `ratio`

Opacity of the cube faces (100% = opaque, 50% = semi-transparent)

Default: `100%`

shadow `boolean`

Draw the footprint on the ground beneath the stack

Default: `false`

disposition `str`

Layout of the 2D grids: "3d" (projected around) or "cote-a-cote" (next to the 3D view)

Default: `"3d"`

box-args `dictionary`

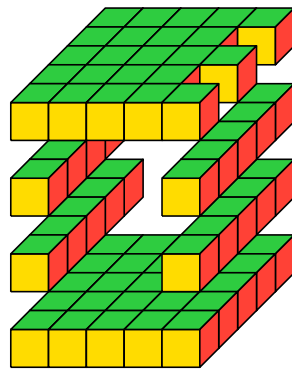
Arguments to pass to the box(es)

Default: `()`

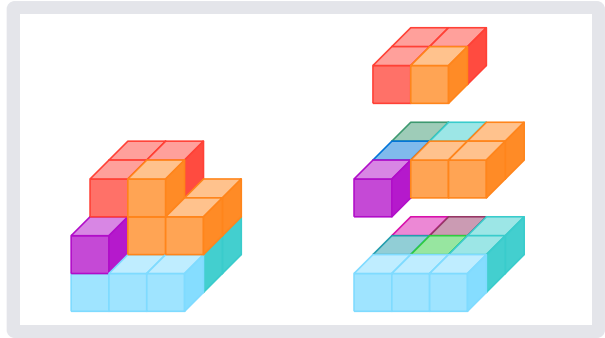
building

Construct of small cubes with mixed colors: each number corresponds to the position in the color list (16 max: 1 to 9 and a to g + t for 3 colors). "0" and "o" represent a hole, "x" uses the first color. Cubes are given as a list of lists: from front to back using a number/string such as "5432a", then layer by layer btt, with lists ordered from left to right.

```
#building((
  ("ttttt","ttttt","ttttt","ttttt"),
  // Leftmost slice, from bottom to top
  ("ttttt","oooooo","oooooo","ttttt"),
  ("ttttt","oooooo","oooooo","ttttt"),
  ("ttttt","oooooo","oooooo","ttttt"),
  ("ttttt","ttttt","ttttt","totot")
  // Rightmost slice, same arrangement
), y-spread:1)
```



```
#building(((578,"21b",44),
("59a",63,64), (533,66,0),))
#h(3em)
#building(((578,"21b",44),
("59a",63,64), (533,66,0),),
y-spread:1.5)
```



z:0 for front view, y:0, z:(0,-1) for top view, x:0, z:(-1,0) for right view

Parameters

```
building(
  ..a: array arguments,
  color-list: array,
  x: array,
  y: array,
  z: array,
  thickness: int float ratio,
  border-stroke: boolean color,
  _rev,
  _dessous,
  _back,
  light: float ratio array,
  draw-scale: float ratio,
  iso: boolean int,
  text: str content,
  x-spread: float int,
  y-spread: float int,
  z-spread: float int,
  box-args: dictionary
) -> content
```

..a array or arguments

List of list of numbers corresponding to the colors back-to-front, bottom to top, left-to-right

color-list array

List of up to 16 colors

Default: typst16

x array

Direction of the first coordinate, default (1,0)

Default: (1,0)

y array

Direction of the second coordinate, default (0,1)

Default: (0,1)

z array

Direction of the third coordinate, default (-0.5,-0.5)

Default: (-.5, -.5)

thickness int or float or ratio

Stroke multiplier (from 1pt), default 60%, i.e. 0.6pt

Default: 60%

border-stroke boolean or color

Lines in black, default false

Default: false

light float or ratio or array

light:20% -> how much to lighten when using a single color

Default: 20%

draw-scale float or ratio

Changes the edge size of a base cube

Default: .5

iso boolean or int

Allows different perspectives:

- true = isometric representation,
- 1 = tikz3d-fr view,
- 2 = first ProfCollege view,
- 3 = second ProfCollege view,
- otherwise = default CeTZ view

Default: false

text `str` or `content`

Text to place below the stack (for the views)

Default: `none`

x-spread `float` or `int`

Allows the generated stack to be “exploded” to the right to make visualization by “slices” easier

Default: `0`

y-spread `float` or `int`

Allows the generated stack to be “exploded” upward to make visualization by “layers” easier

Default: `0`

z-spread `float` or `int`

Allows the generated stack to be “exploded” forward to make visualization by “slices” easier

Default: `0`

box-args `dictionary`

Arguments to pass to the box

Default: `()`

above-view

Top view for a building. Change text to “Top view” or “”

Parameters

```
above-view(  
  ..it: arguments ,  
  text: str content  
) -> content
```

..it `arguments`

Arguments for building

text `str` or `content`

View name

Default: `"Vue du dessus"`

right-view

Right view for a building. Change text to “Right view”

Parameters

```
right-view(  
  ..it: arguments ,  
  text: str content  
) -> content
```

..it arguments

Arguments for building

text str or content

View name

Default: "Vue de droite"

front-view

Front view for a building. Change text to “Front view”

Parameters

```
front-view(  
  ..it: arguments ,  
  text: str content  
) -> content
```

..it arguments

Arguments for building

text str or content

View name

Default: "Vue de face"

below-view

Bottom view for a building. Change text to “Bottom view”

Parameters

```
below-view(  
  ..it: arguments ,  
  text: str content  
) -> content
```

```
..it arguments
```

Arguments for building

```
text str or content
```

View name

Default: "Vue du dessous"

left-view

Left view for a building. Change text to “Left view”

Parameters

```
left-view(  
    ..it: arguments ,  
    text: str content  
) -> content
```

```
..it arguments
```

Arguments for building

```
text str or content
```

View name

Default: "Vue de gauche"

back-view

Back view for a building. Change text to “Back view”

Parameters

```
back-view(  
    ..it: arguments ,  
    text: str content  
) -> content
```

```
..it arguments
```

Arguments for building

text `str` or `content`

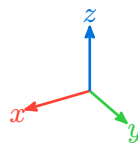
View name

Default: "Vue arrière"

thetaphi

Function that sets the 3D viewing angle for a `cetz.canvas` using polar (θ) and azimuthal (φ) angles. You have to pass it directly to the canvas, without forgetting to spread.

```
#canvas(..thetaphi(theta:60, phi:150), {  
  import draw:*  
  repere  
})
```



Parameters

```
thetaphi(  
  theta: int angle,  
  phi: int angle  
) -> dictionary
```

theta `int` or `angle`

Polar angle θ

Default: 70

phi `int` or `angle`

Azimuthal angle φ

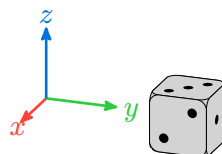
Default: 110

dice

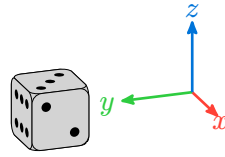
Function used to draw a 3D die.

Warning, using `dice` in a canvas overwrites the transformation matrix!

```
#canvas({  
  import draw:*  
  dice(standalone:false, origin: (0,2),  
    front:2, top:3, s:.8, vue: "D")  
  repere  
})
```



```
#canvas({
  import draw:*
  dice(standalone: false, origin: (0,2),
    front: 2, top:3, s:.8, vue: "G")
  repere
})
```



Parameters

```
dice(
  standalone: boolean,
  front: int,
  top: int,
  s: int float ratio,
  theta: int float,
  phi: int float,
  vue: str,
  color: color,
  dot-color: color,
  origin: array,
  box-args: dictionary,
  opacity: ratio
) -> content function
```

standalone `boolean`

true if alone, false if already inside a canvas

Default: `true`

front `int`

Number of spots on the front face

Default: `1`

top `int`

Number of spots on the top face

Default: `2`

s `int` or `float` or `ratio`

Drawing scale

Default: `1.0`

theta `int` or `float`

Theta angle of the spherical coordinates

Default: `70`

phi `int` or `float`

Phi angle of the spherical coordinates

Default: `110`

vue `str`

“D” for right view or “G” for left view

Default: `"D"`

color `color`

Die color, light gray by default

Default: `rgb("d3d3d3")`

dot-color `color`

Spot color

Default: `black`

origin `array`

Position of the die's origin

Default: `(0,0,0)`

box-args `dictionary`

Additional arguments for the box

Default: `()`

opacity `ratio`

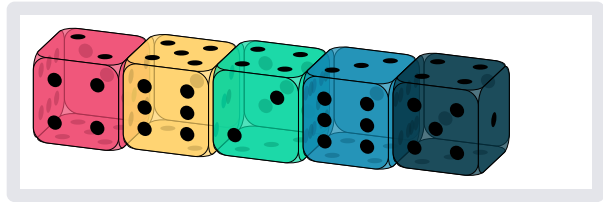
Opacité des dés

Default: `100%`

throws

Random or non-random roll of n dice.

```
#throws(5, yams: true, colors:
(rgb("ef476f"), rgb("ffd166"),
rgb("06d6a0"), rgb("118ab2"),
rgb("073b4c")), seed: 999, s: 1.2,
yams-h: -.15, opacity: 70%,)
```



Parameters

```
throws(
  n: int,
  list: none array,
  seed: int,
  yams: boolean,
  yams-h: int float,
  espace-h: length,
  s: int float ratio,
  colors: array,
  dot-color: color,
  theta: int float,
  phi: int float,
  vue: str,
  box-args: dictionary,
  opacity: ratio
) -> content
```

n int

Number of dice

list none or array

List of rolls: none by default, otherwise a list of pairs (front face, top face)

Default: none

seed int

Pseudo-random seed

Default: 42

yams boolean

Dice displayed “touching” each other

Default: false

yams-h `int` or `float`

Horizontal offset of the dice in the row when yams is enabled

Default: `.1`

espace-h `length`

Horizontal spacing between dice

Default: `5mm`

s `int` or `float` or `ratio`

Scale factor

Default: `1.0`

colors `array`

List of die colors

Default: `(rgb("d3d3d3"),)`

dot-color `color`

Spot color

Default: `black`

theta `int` or `float`

Theta angle of the spherical coordinates

Default: `70`

phi `int` or `float`

Phi angle of the spherical coordinates

Default: `110`

vue `str`

“D” for right view or “G” for left view

Default: `"D"`

box-args dictionary

Additional arguments for the box

Default: ()

opacity ratio

Opacité des dés

Default: 100%

dice-face

From eric1102 on Discord: draw one side of a die with the possibility to change the fill color, spot fill, and stroke

Parameters

```
dice-face(  
  num: int,  
  fill: none color,  
  spot-fill: auto color,  
  stroke: auto stroke  
) -> content
```

num int

Int from 1 to 6

fill none or color

Color of the dice

Default: none

spot-fill auto or color

Black for a light-colored fill, white for a dark-colored fill, or a chosen color

Default: auto

stroke auto or stroke

Stroke of the dice, if a fill is provided and stroke:auto, same as the fill

Default: auto